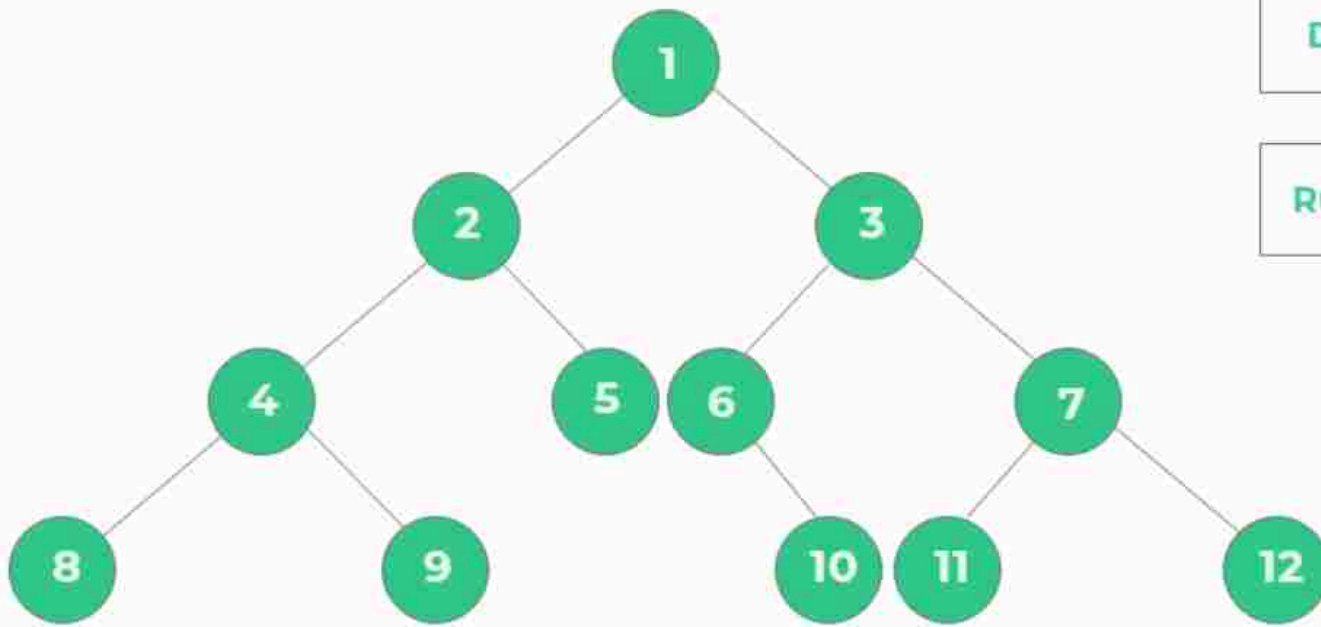




Direction : Anti Clock

Rule : CLR (Center-Left-Right)

PreOrder:

1 2 4 8 9 5 3 6 10 7 11 12

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the tree

Second line indicates the elements of the tree

Output Format:

single line indicates the pre-order list of the given value

Logical Test Cases

Test Case 1

INPUT (STDIN)

7
200 120 80 160 280 240 320

EXPECTED OUTPUT

200 120 80 160 280 240 320

Test Case 2

INPUT (STDIN)

9
34 67 10 90 410 810 40 20 60

EXPECTED OUTPUT

34 10 20 67 40 60 90 410 810

Mandatory Test Cases

Test Case 1

KEYWORD

void preorder(struct node* root)

Test Case 2

KEYWORD

struct node* newNode(int item)

Test Case 3

KEYWORD

struct node

Test Case 4

KEYWORD

struct node *left,*right;

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

280

Test Case 3

NLOC

50

Tree11.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node *left,*right;
7 };
8
9 struct node* newNode(int item) {
10     struct node* temp = (struct node*)malloc(sizeof(struct node));
11     temp->data = item;
12     temp->left = NULL;
13     temp->right = NULL;
14     return temp;
15 }
16
17 struct node* insert(struct node* node, int data) {
18     if (node == NULL) return newNode(data);
19     if (data < node->data)
20         node->left = insert(node->left, data);
21     else if (data > node->data)
22         node->right = insert(node->right, data);
23     return node;
24 }
```

```
26 void preorder(struct node* root) {
27     if (root != NULL) {
28         printf("%d ", root->data);
29         preorder(root->left);
30         preorder(root->right);
31     }
32 }
33
34 int main() {
35     int n, val;
36     if (scanf("%d", &n) != 1) return 0;
```

```
37 struct node* root = NULL;
38 for (int i = 0; i < n; i++) {
39     if (scanf("%d", &val) == 1) {
40         root = insert(root, val);
41     }
42 }
43 preorder(root);
44 printf("\n");
45 return 0;
46}
47
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
7
```

```
200 120 80 160 280 240 320
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree11.exe
```

```
200 120 80 160 280 240 320
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
9
```

```
34 67 10 90 410 810 40 20 60
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree11.exe
```

```
34 10 20 67 40 60 90 410 810
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Question description

There are three common ways to traverse the nodes of a binary tree:

- *Preorder*: First process the root, then the left subtree, and finally the right subtree.
- *Inorder*: First process the left subtree, then the root, and finally the right subtree.
- *Postorder*: First process the left subtree, then the right subtree, and finally the root.

There is a binary tree of n nodes with distinct labels. You are given the preorder and inorder traversals of the tree, and your task is to determine its postorder traversal.

Constraints

- $1 \leq n \leq 10^5$

Input

The first input line has an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.

After this, there are two lines describing the preorder and inorder traversals of the tree. Both lines consist of n integers.

You can assume that the input corresponds to a binary tree.

Output

Print the postorder traversal of the tree.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
5 3 2 1 4
3 5 1 2 4

EXPECTED OUTPUT

3 1 4 2 5

Test Case 2

INPUT (STDIN)

6
6 5 3 2 1 4
6 3 5 1 2 4

EXPECTED OUTPUT

3 1 4 2 5 6

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=1;i<=n;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

390

Test Case 3

NLOC

41

```
Tree12.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 100005
5
6 int pre[MAXN];
7 int in[MAXN];
8 int pos[MAXN];
9
10 void solve(int pre_l, int in_l, int len) {
11     if (len <= 0) return;
12     int root = pre[pre_l];
13     int root_pos = pos[root];
14     int left_len = root_pos - in_l;
15     int right_len = len - 1 - left_len;
16
17     solve(pre_l + 1, in_l, left_len);
18     solve(pre_l + 1 + left_len, root_pos + 1, right_len);
19
20 }
21
22 int main() {
23     int n, i;
24     if (scanf("%d", &n) != 1) return 0;
25     for(i=1;i<=n;i++) {
```

main | CH.SC.U4CSE25225 | Tree12.c

Ln 36, Col 1 Spaces: 4 UTF-8 CRLF C

```
26     scanf("%d", &pre[i]);
27 }
28 for(i=1;i<=n;i++) {
29     scanf("%d", &in[i]);
30     pos[in[i]] = i;
31 }
```

```
32 solve(1, 1, n);  
33 printf("\n");  
34 return 0;  
35}  
36
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

5 3 2 1 4

3 5 1 2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree12.exe

3 1 4 2 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

6

6 5 3 2 1 4

6 3 5 1 2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree12.exe

3 1 4 2 5 6

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates $(1, 1)$, and the lower-right square has coordinates (n, n) .

Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest?

Constraints

- $1 \leq n \leq 1000$
- $1 \leq q \leq 2 \cdot 10^5$
- $1 \leq y_1 \leq y_2 \leq n$
- $1 \leq x_1 \leq x_2 \leq n$

Input

The first input line has two integers n and q : the size of the forest and the number of queries.

Then, there are n lines describing the forest. Each line has n characters: `.` is an empty square and `*` is a tree.

Finally, there are q lines describing the queries. Each line has four integers y_1, x_1, y_2, x_2 corresponding to the corners of a rectangle.

Output

Print the number of trees inside each rectangle.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) <pre>4 3 .*. *.* **.</pre> 2 2 3 4 3 1 3 1 1 1 2 2 EXPECTED OUTPUT <pre>3 1 2</pre>	INPUT (STDIN) <pre>4 3 .*.* *.* **.* ***.</pre> 1 2 3 4 3 2 3 4 1 4 2 3 EXPECTED OUTPUT <pre>6 2 0</pre>

▼ Mandatory Test Cases

Test Case 1
KEYWORD
<code>for(i=1;i<=n;i++)</code>

▼ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
8

Test Case 2
TOKEN COUNT
310

Test Case 3
NLOC
28

```
1 #include <stdio.h>
2
3 #define MAXN 1005
4
5 int pref[MAXN][MAXN];
6 char row[MAXN];
7
8 int main() {
9     int n, q, i, j;
10    if (scanf("%d %d", &n, &q) != 2) return 0;
11    for(i=1;i<=n;i++) {
12        scanf("%s", row);
13        for (j = 1; j <= n; j++) {
14            int val = (row[j - 1] == '*') ? 1 : 0;
15            pref[i][j] = pref[i - 1][j] + pref[i][j - 1] - pref[i - 1][j - 1] + val;
16        }
17    }
18    for (int k = 0; k < q; k++) {
19        int y1, x1, y2, x2;
20        if (scanf("%d %d %d %d", &y1, &x1, &y2, &x2) == 4) {
21            if (y1 > y2 || x1 > x2) {
22                printf("0\n");
23            } else {
24                int ans = pref[y2][x2] - pref[y1 - 1][x2] - pref[y2][x1 - 1] + pref[y1 - 1][x1 - 1];
25                printf("%d\n", ans);

```

```
26     }  
27     }  
28 }
```

```
29     return 0;  
30}  
31
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4 3

. * . .

* . **

** . .

2 2 3 4

3 1 3 1

1 1 2 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree13.exe

3

1

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4 3

. * . *

* . **

** . *

*** .

1 2 3 4

3 2 3 4

1 4 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree13.exe

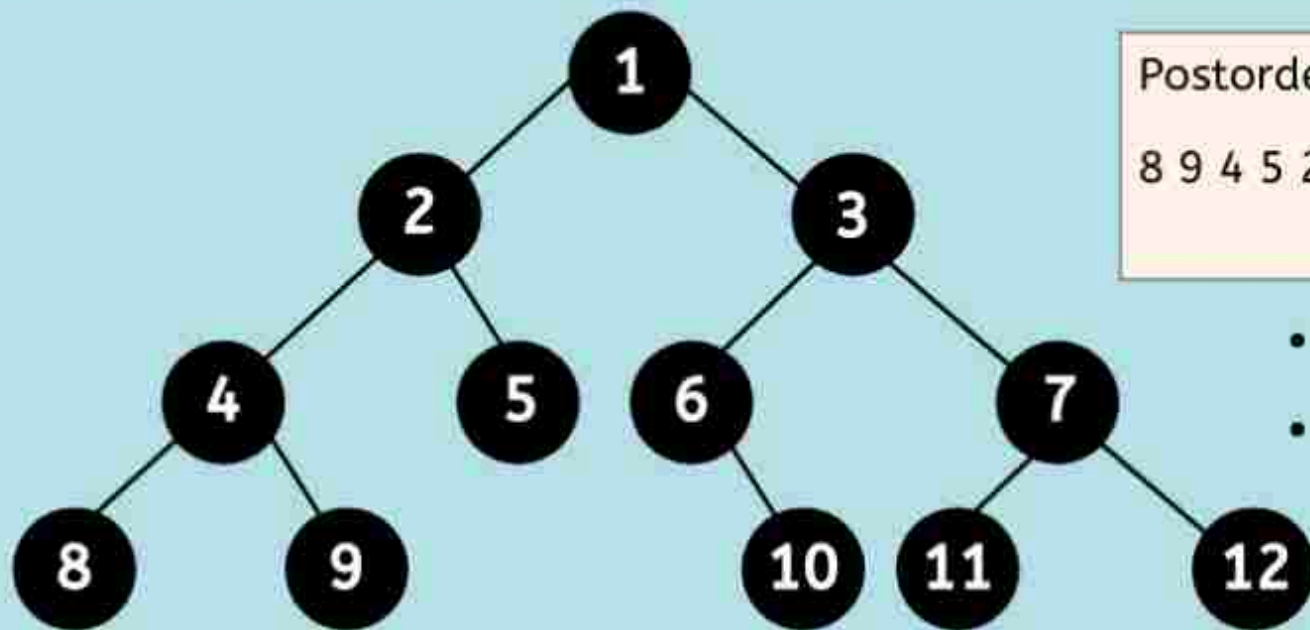
6

2

0

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Tree 1	Question Information	Level 1	Challenge 64
Question description Darsh, Ratik, Swathy are good friends, They are studying Pre-final year B.E. in reputed institution. Swathy's uncle was a DS teacher in school of computing technologies. He asks to make an application for identifying postorder Traversal of the Tree You have to do the work for the development of the code of their thinking. Functional Description					
CH.SC.U4CSE25225 - M ASHWIN Department of Computer Science & Engineering (CSE) 2026					



Postorder

8 9 4 5 2 10 6 11 12 7 3

- Anti clockwise
- Left-Center-Right

Constraints

$T \leq 100$

$1 \leq n \leq 1000$

$0 \leq k < n$

Input Format

The first line of input consists of the number of test cases T . Then T lines follow each describing a test case. The first line of each test case consists of a number n . Then follow n lines each describing the friendship of a student. Each line starts with a number k telling the number of friends the i th person has. Then follow k integers separated by space which tells the index (1 based) of his friends. All these integers on the i th line are greater than i itself. (See sample for clarity).

Output Format

For each test case, if it is possible to divide the class into teams, then print a line of L's and R's denoting which team that student is a part of. If it is impossible to divide the class then print -1 instead. If there are multiple solutions print the one that would come first in the dictionary.

INPUT (STDIN)

7
90 70 50 75 130 110 150

EXPECTED OUTPUT

50 75 70 110 150 130 90

INPUT (STDIN)

9
50 23 56 89 43 75 130 110 150

EXPECTED OUTPUT

43 23 75 110 150 130 89 56 50

▼ Mandatory Test Cases

Test Case 1

KEYWORD

struct node

Test Case 2

KEYWORD

struct node *left,*right;

Test Case 3

KEYWORD

struct node* newNode(int item)

Test Case 4

KEYWORD

void postorder(struct node* root)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

290

Test Case 3

NLOC

50

Tree14.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node {
5     int data;
6     struct node *left,*right;
7 };
8
9 struct node* newNode(int item) {
10     struct node* temp = (struct node*)malloc(sizeof(struct node));
11     temp->data = item;
12     temp->left = NULL;
13     temp->right = NULL;
14     return temp;
15 }
16
17 struct node* insert(struct node* node, int data) {
18     if (node == NULL) return newNode(data);
19     if (data < node->data)
20         node->left = insert(node->left, data);
21     else if (data > node->data)
22         node->right = insert(node->right, data);
23     return node;
24 }
```

```
26 void postorder(struct node* root) {  
27     if (root != NULL) {  
28         postorder(root->left);  
29         postorder(root->right);  
30         printf("%d ", root->data);  
31     }  
32 }  
33  
34 int main() {  
35     int n, val;  
36     if (scanf("%d", &n) != 1) return 0;
```

```
37 struct node* root = NULL;
38 for (int i = 0; i < n; i++) {
39     if (scanf("%d", &val) == 1) {
40         root = insert(root, val);
41     }
42 }
43 postorder(root);
44 printf("\n");
45 return 0;
46}
47
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
7
```

```
90 70 50 75 130 110 150
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree14.exe
```

```
50 75 70 110 150 130 90
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
9
```

```
50 23 56 89 43 75 130 110 150
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree14.exe
```

```
43 23 75 110 150 130 89 56 50
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

Question description

Given an array of n integers, your task is to process q queries of the form: what is the minimum value in range $[a,b]$?

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$
- $1 \leq a \leq b \leq n$

Input

The first input line has two integers n and q : the number of values and queries.

The second line has n integers $x_1, x_2, \dots, x_n$: the array values.

Finally, there are q lines describing the queries. Each line has two integers a and b : what is the minimum value in range $[a,b]$?

Output

Print the result of each query.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
8 4
3 2 4 5 1 1 5 3
2 4
5 6
1 8
3 3
```

EXPECTED OUTPUT

```
2
1
1
4
```

Test Case 2

INPUT (STDIN)

```
10 4
13 12 14 51 11 1 5 3 4 6
3 4
3 6
4 8
1 3
```

EXPECTED OUTPUT

```
14
1
1
12
```

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
void build(int *aa,int k,int l,int r)
```

Test Case 2

KEYWORD

```
int query(int k,int l,int r,int ql,int qr)
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

390

Test Case 3

NLOC

40

```
Tree15.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 100005
5 #define INF 2147483647
6
7 int tree[4 * MAXN];
8
9 void build(int *aa, int k, int l, int r) {
10     if (l == r) {
11         tree[k] = aa[l];
12         return;
13     }
14     int mid = (l + r) / 2;
15     build(aa, k * 2, l, mid);
16     build(aa, k * 2 + 1, mid + 1, r);
17     tree[k] = (tree[k * 2] < tree[k * 2 + 1]) ? tree[k * 2] : tree[k * 2 + 1];
18 }
19
20 int query(int k, int l, int r, int ql, int qr) {
21     if (qr < l || r < ql) return INF;
22     if (ql <= l && r <= qr) return tree[k];
23     int mid = (l + r) / 2;
24     int a = query(k * 2, l, mid, ql, qr);
25     int b = query(k * 2 + 1, mid + 1, r, ql, qr);
```

```
26     return (a < b) ? a : b;
27 }
28
29 int main() {
30     int n, q;
31     if (scanf("%d %d", &n, &q) != 2) return 0;
32     int *aa = (int*)malloc(n * sizeof(int));
33     for (int i = 1; i <= n; i++) scanf("%d", &aa[i]);
34     build(aa, 1, 1, n);
35     for (int i = 0; i < q; i++) {
36         int a, b;
```

```
37     if (scanf("%d %d", &a, &b) == 2) {
38         if (a > b) {
39             int t = a;
40             a = b;
41             b = t;
42         }
43         printf("%d\n", query(1, 1, n, a, b));
44     }
45 }
46 return 0;
47 }
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

8 4

3 2 4 5 1 1 5 3

2 4

5 6

1 8

3 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree15.exe

2

1

1

4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

10 4

13 12 14 51 11 1 5 3 4 6

3 4

3 6

4 8

1 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree15.exe

14

1

1

12

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director. Your task is to process q queries of the form: who is employee x 's boss k levels higher up in the hierarchy?

Input

The first input line has two integers n and q : the number of employees and queries. The employees are numbered $1, 2, \dots, n$, and employee 1 is the general director. The next line has $n-1$ integers $e_2, e_3, \dots, e_n$: for each employee $2, 3, \dots, n$ their boss. Finally, there are q lines describing the queries. Each line has two integers x and k : who is employee x 's boss k levels higher up?

Output

Print the answer for each query. If such a boss does not exist, print -1 .

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq e_i \leq i-1$
- $1 \leq x \leq n$
- $1 \leq k \leq n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
1 3 2 2
4 2
1 2
1 3
```

EXPECTED OUTPUT

```
1
-1
-1
```

Test Case 2

INPUT (STDIN)

```
5 3
1 1 3 3
4 1
4 2
4 3
```

EXPECTED OUTPUT

```
3
1
-1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int j)
```

▼ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
2

Test Case 2
TOKEN COUNT
420

Test Case 3
NLOC
50

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 200005
5 #define LOG 18
6
7 int up[MAXN][LOG];
8 int depth[MAXN];
9
10 int main() {
11     int n, q;
12     if (scanf("%d %d", &n, &q) != 2) return 0;
13
14     for (int i = 2; i <= n; i++) {
15         int p;
16         scanf("%d", &p);
17         up[i][0] = p;
18         depth[i] = depth[p] + 1;
19         for (int j = 1; j < LOG; j++) {
20             up[i][j] = up[up[i][j-1]][j-1];
21         }
22     }
23
24     for (int i = 0; i < q; i++) {
25         int x, k;
```

Ln 39, Col 1 Spaces: 4 UTF-8 CRLF C

```
26     scanf("%d %d", &x, &k);
27     if (k > depth[x]) {
28         printf("-1\n");
29     } else {
30         for (int j = LOG - 1; j >= 0; j--) {
31             if (k & (1 << j)) {
32                 x = up[x][j];
```

```
33         }
34     }
35     printf("%d\n", x);
36 }
37 }
38 return 0;
39 }
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 3

1 3 2 2

4 2

1 2

1 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree16.exe

1

-1

-1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3

1 1 3 3

4 1

4 2

4 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree16.exe

3

1

-1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

DS	Session	Tree 1	Question Information	Level 1	Challenge 67
Question description The sam is enjoying a wonderful vacation in Byteland. What makes the sam impressed the most is the road system of the country. Byteland has N cities numbered 1 through N. City 1 is the capital of Byteland. The country also has $N-1$ bidirectional roads connecting the cities. The i-th road connects two different cities u_i and v_i. In this road system, people can travel between every pair of different cities by going through exactly one path of roads. The roads are arranged in such a way that people can distinguish two cities only when both cities have different number of roads connected to it. Such two cities will be considered similar. For example, city A is similar to the capital if the number of roads connected to city A is equal to the number of roads connected to the capital. On each day during the vacation, the sam wants to have a trip as follows. He chooses two cities A and B such that the sam will visit city B if he goes from A to the capital using the shortest path. Then, the sam will visit the cities on the shortest path from A to B through this path. Please note that A may be equal to B; that means the sam will enjoy the day in a single city. The sam does not want to have similar trips. Two trips are considered similar if and only if they both have the same number of visited cities and for each i, the i-th city visited in one trip is similar to the i-th city visited in the other trip. The sam wants to have as many different, namely not similar, trips as possible. Help him count the maximum number of possible trips such that no two of them are similar. Input Format The first line of the input contains a single integer N. The i-th line of next $N-1$ lines contains two space-separated integers u_i and v_i, denoting the i-th road. Output Format Output a single line containing the maximum number of different trips.					

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3
2 1
3 1

EXPECTED OUTPUT

3

Test Case 2

INPUT (STDIN)

4
2 1
3 1
2 4

EXPECTED OUTPUT

5

▼ Mandatory Test Cases

Test Case 1

KEYWORD

struct state

Test Case 2

KEYWORD

const int MAXL=200005;

Test Case 3

KEYWORD

state st[MAXL]

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

700

Test Case 3

NLOC

67

```
Tree17.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXL 200005
5 #define HASHSIZE (1 << 21)
6 #define HASHMASK (HASHSIZE - 1)
7
8 int deg[MAXL];
9 int head[MAXL], to[2 * MAXL], nxt[2 * MAXL], ecnt;
10 long long keys[HASHSIZE];
11 int vals[HASHSIZE];
12 int stack_node[MAXL];
13 int stack_parent[MAXL];
14 int stack_edge[MAXL];
15 int path[MAXL];
16
17 void add_edge(int a, int b) {
18     to[ecnt] = b;
19     nxt[ecnt] = head[a];
20     head[a] = ecnt++;
21 }
22
23 int get_child(int parent, int d) {
24     long long key = (long long)parent * 1000003LL + d;
25     int h = (unsigned)key & HASHMASK;
```

```

26 while (keys[h] != 0 && keys[h] != key) h = (h + 1) & HASHMASK;
27 if (keys[h] == 0) return -1;
28 return vals[h];
29}
30
31void set_child(int parent, int d, int child) {
32     long long key = (long long)parent * 1000003LL + d;
33     int h = (unsigned)key & HASHMASK;
34     while (keys[h] != 0 && keys[h] != key) h = (h + 1) & HASHMASK;
35     keys[h] = key;
36     vals[h] = child;
37}
38
39int main() {
40     int n, i;
41     if (scanf("%d", &n) != 1) return 0;
42     for (i = 0; i < HASHSIZE; i++) keys[i] = 0;

```

```

43  ecnt = 0;
44  for (i = 1; i <= n; i++) head[i] = -1;
45  for (i = 0; i < n - 1; i++) {
46      int u, v;
47      scanf("%d %d", &u, &v);
48      add_edge(u, v);
49      add_edge(v, u);
50  }
51  for (i = 1; i <= n; i++) {
52      int c = 0;
53      for (int e = head[i]; e != -1; e = nxt[e]) c++;
54      deg[i] = c;
55  }

```

main | CH.SC.U4CSE25225 | Tree17.c

Ln 107, Col 1 Spaces: 4 UTF-8 CRLF C

```

56
57  long long ans = 0;
58  int sp = 0;
59  int path_top = 0;

```

```
60  stack_node[sp] = 1;
61  stack_parent[sp] = 0;
62  stack_edge[sp] = head[1];
63  path[path_top++] = deg[1];
64  sp++;
65  int node = 0;
66  for (i = path_top - 1; i >= 0; i--) {
67      int nx = get_child(node, path[i]);
68      if (nx == -1) {
69          nx = (int)(ans + 1);
70          ans++;
71          set_child(node, path[i], nx);
72      }
73      node = nx;
74  }
75
```

```
76 while (sp > 0) {
77     int u = stack_node[sp - 1];
78     int par = stack_parent[sp - 1];
79     int e = stack_edge[sp - 1];
80     if (e == -1) {
81         sp--;
82         path_top--;
83         continue;
84     }
85     stack_edge[sp - 1] = nxt[e];
86     int v = to[e];
87     if (v == par) continue;
88     path[path_top++] = deg[v];
89     node = 0;
90     for (i = path_top - 1; i >= 0; i--) {
91         int nx = get_child(node, path[i]);
```

```
92         if (nx == -1) {
93             nx = (int)(ans + 1);
94             ans++;
95             set_child(node, path[i], nx);
96         }
97         node = nx;
98     }
99     stack_node[sp] = v;
100    stack_parent[sp] = u;
101    stack_edge[sp] = head[v];
102    sp++;
103 }
104
105 printf("%lld\n", ans);
106 return 0;
107 }
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3

2 1

3 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree17.exe

3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4

2 1

3 1

2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree17.exe

5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

A company has n employees with certain salaries. Your task is to keep track of the salaries and process queries.

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq p_i \leq 10^9$
- $1 \leq k \leq n$
- $1 \leq x \leq 10^9$
- $1 \leq a \leq b \leq 10^9$

Input

The first input line contains two integers n and q : the number of employees and queries. The employees are numbered $1, 2, \dots, n$.

The next line has n integers $p_1, p_2, \dots, p_n$: each employee's salary.

After this, there are q lines describing the queries. Each line has one of the following forms:

- $! k x$: change the salary of employee k to x
- $? a b$: count the number of employees whose salary is between $a \dots b$

Output

Print the answer to each $?$ query.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3

EXPECTED OUTPUT

3
2

Test Case 2

INPUT (STDIN)

5 3
31 17 12 12 15
? 12 31
! 13 6
? 2 13

EXPECTED OUTPUT

5
3

▼ Mandatory Test Cases

Test Case 1

KEYWORD

int compare(const void
*a,const void *b)

Test Case 2

KEYWORD

void update(int i,int
n,int x)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

575

Test Case 3

NLOC

65

```
Tree18.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 400005
5
6 long long arr[MAXN];
7 long long vals[MAXN];
8 int bit[MAXN];
9 int n, q, sz;
10
11 char qtype[MAXN];
12 long long qa1[MAXN];
13 long long qa2[MAXN];
14
15 int cmp(const void *a, const void *b) {
16     long long x = *(const long long *)a;
17     long long y = *(const long long *)b;
18     return x < y ? -1 : (x > y ? 1 : 0);
19 }
20
21 int index_of(long long x) {
22     int lo = 1, hi = sz, ans = sz + 1;
23     while (lo <= hi) {
24         int mid = (lo + hi) / 2;
25         if (vals[mid] >= x) {
```

```
26         ans = mid;
27         hi = mid - 1;
28     } else {
29         lo = mid + 1;
30     }
31 }
32 return ans;
33}
34
35int upper_of(long long x) {
36     int lo = 1, hi = sz, ans = 0;
37     while (lo <= hi) {
38         int mid = (lo + hi) / 2;
39         if (vals[mid] <= x) {
40             ans = mid;
41             lo = mid + 1;
42         } else {
43             hi = mid - 1;
```

```
44     }
45 }
46 return ans;
47}
48
49void update(int i, int x) {
50     while (i <= sz) {
51         bit[i] += x;
52         i += i & (-i);
53     }
54}
55
56int query(int i) {
57     int res = 0;
```

main | CH.SC.U4CSE25225 | Tree18.c

Ln 114, Col 1 Spaces: 4 UTF-8 CRLF C

```
59     res += bit[i];
60     i -= i & (-i);
61 }
```

```

62     return res;
63 }
64
65 int main() {
66     int i;
67     char op;
68     if (scanf("%d %d", &n, &q) != 2) return 0;
69     for (i = 1; i <= n; i++) {
70         scanf("%lld", &arr[i]);
71         vals[sz++] = arr[i];
72     }
73     for (i = 0; i < q; i++) {
74         scanf(" %c", &op);
75         qtype[i] = op;
76         if (op == '!') {
77             int k;
78             long long x;
79             scanf("%d %lld", &k, &x);

```

```
80         qa1[i] = k;
81         qa2[i] = x;
82         vals[sz++] = x;
83     } else {
84         long long a, b;
85         scanf("%lld %lld", &a, &b);
86         qa1[i] = a;
87         qa2[i] = b;
88     }
89 }
90
91 qsort(vals, (size_t)sz, sizeof(long long), cmp);
92 int uniq = 0;
93 int j = 0;
94 while (j < sz) {
95     int k = j;
96     while (k < sz && vals[k] == vals[j]) k++;
97     vals[uniq++] = vals[j];
```

```

98     j = k;
99 }
100 sz = uniq;
101
102 for (i = 1; i <= n; i++) update(index_of(arr[i]), 1);
103 for (i = 0; i < q; i++) {
104     if (qtype[i] == '!') {
105         int k = (int)qa1[i];
106         update(index_of(arr[k]), -1);
107         arr[k] = qa2[i];
108         update(index_of(arr[k]), 1);
109     } else {
110         printf("%d\n", query(upper_of(qa2[i])) - query(index_of(qa1[i]) - 1));
111     }
112 }
113 return 0;
114 }

```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 3

3 7 2 2 5

? 2 3

! 3 6

? 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree18.exe

3

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3

31 17 12 12 15

? 12 31

! 13 6

? 2 13

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree18.exe

5

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Output

The first line contains only one integer h , the maximum number of edges Mohana and John can add.

Each of the next h lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) — the edge you add each time.

If there are multiple correct answers, you can print any one of them.

Logical Test Cases

Test Case 1

INPUT (STDIN)

5 3 2
5 4
2 1
4 3
4 3
1 4

EXPECTED OUTPUT

1
1 5

Test Case 2

INPUT (STDIN)

5 3 2
5 3
2 3
2 3
2 3
2 4

EXPECTED OUTPUT

1
1 2
4 5

Mandatory Test Cases

Test Case 1

KEYWORD

while(m1--)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

500

Test Case 3

NLOC

30

```
1 #include <stdio.h>
2
3 #define MAXN 100005
4
5 int par1[MAXN];
6 int par2[MAXN];
7 int eu[MAXN];
8 int ev[MAXN];
9
10 void init(int par[], int n) {
11     int i;
12     for (i = 1; i <= n; i++) par[i] = i;
13 }
14
15 int find(int par[], int x) {
16     while (par[x] != x) {
17         par[x] = par[par[x]];
18         x = par[x];
19     }
20     return x;
21 }
22
23 void unite(int par[], int a, int b) {
24     a = find(par, a);
25     b = find(par, b);
```

```
26     if (a != b) par[a] = b;
```

```
27 }
```

```
28
```

```
29 int main() {
```

main | CH.SC.U4CSE25225 | Tree19.c

Ln 57, Col 1 Spaces: 4 UTF-8 CRLF C

```
30     int n, m1, m2, i;
```

```
31     int h = 0;
```

```
32     if (scanf("%d %d %d", &n, &m1, &m2) != 3) return 0;
```

```
33     init(par1, n);
```

```
34     init(par2, n);
```

```
35     for (i = 0; i < m1; i++) {
```

```
36         int u, v;
```

```
37     scanf("%d %d", &u, &v);
38     unite(par1, u, v);
39 }
40 for (i = 0; i < m2; i++) {
41     int u, v;
42     scanf("%d %d", &u, &v);
43     unite(par2, u, v);
44 }
45 for (i = 2; i <= n; i++) {
46     if (find(par1, i) != find(par1, 1) && find(par2, i) != find(par2, 1)) {
47         unite(par1, i, 1);
```

```
48         unite(par2, i, 1);
49         eu[h] = 1;
50         ev[h] = i;
51         h++;
52     }
53 }
54 printf("%d\n", h);
55 for (i = 0; i < h; i++) printf("%d %d\n", eu[i], ev[i]);
56 return 0;
57}
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 3 2

5 4

2 1

4 3

4 3

1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree19.exe

1

1 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3 2

5 3

2 3

2 3

2 3

2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree19.exe

1

1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

Question description

You are given a list consisting of n integers. Your task is to remove elements from the list at given positions, and report the removed elements.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$
- $1 \leq p_i \leq n - i + 1$

Input

The first input line has an integer n : the initial size of the list. During the process, the elements are numbered $1, 2, \dots, k$ where k is the current size of the list.

The second line has n integers $x_1, x_2, \dots, x_n$: the contents of the list.

The last line has n integers $p_1, p_2, \dots, p_n$: the positions of the elements to be removed.

Output

Print the elements in the order they are removed.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
2 6 1 4 2
3 1 3 1 1

EXPECTED OUTPUT

1 2 2 6 4

Test Case 2

INPUT (STDIN)

6
7 2 6 1 4 2
11 3 1 3 1 1

EXPECTED OUTPUT

2 6 7 4 2 1

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void build(int k,int
l,int r)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

330

Test Case 3

NLOC

37

Tree110.c +

```
1 #include <stdio.h>
2
3 #define MAXN 200005
4
5 int arr[MAXN];
6 int bit[MAXN];
7 int n;
8
9 void update(int i, int x) {
10     while (i <= n) {
11         bit[i] += x;
12         i += i & (-i);
13     }
14 }
15
16 int find_kth(int k) {
17     int idx = 0;
18     int step = 1;
19     while (step * 2 <= n) step *= 2;
20     for (; step >= 1; step /= 2) {
21         int nx = idx + step;
22         if (nx <= n && bit[nx] < k) {
23             k -= bit[nx];
24         }
25     }
```

```
26     }
27     return idx + 1;
28 }
29
30 int main() {
31     int i;
32     if (scanf("%d", &n) != 1) return 0;
33     for (i = 1; i <= n; i++) scanf("%d", &arr[i]);
34     for (i = 1; i <= n; i++) update(i, 1);
35     for (i = 0; i < n; i++) {
```

```
36     int p;  
37     scanf("%d", &p);  
38     int idx = find_kth(p);  
39     printf("%d ", arr[idx]);  
40     update(idx, -1);  
41 }  
42 printf("\n");  
43 return 0;  
44 }
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

2 6 1 4 2

3 1 3 1 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree110.exe

1 2 2 6 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

6

7 2 6 1 4 2

11 3 1 3 1 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree110.exe

0 6 7 4 2 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>